The Sharding Resurrection

Matrix: A Scalable Sharding Protocol With Transaction Simulation

Index

Decipher

Performance Is No Longer a Bottleneck

Now, 100,000+ TPS with Sub-second Latency is Achievable in a Distributed Network

Development of Consensus Algorithm

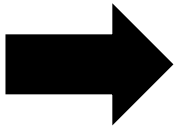
- DAG-based BFT Consensus
- Consensus Pipelining

Development of Data Dissemination

- Separating Data Dissemination from Consensus
- Data Availability Sampling
- Structured Propagation

Development of State Management

- High-Performance Verifiable State Store
- Optimistic Concurrency Control



100,000+ TPS

Latency < 1s

Decipher

Scalability Is a Bottleneck

Performance Is Solved, It's Time to Consider the Scalability

No More Optimizing Tweaks Left on Consensus

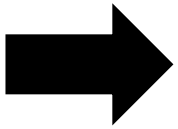
- Modern Consensus Algorithms Are at Near-Optimum

Trade-off Between Performance and Network Size

- Expanded Validator Set → Compromise Performance
- Limited Validator Set → Compromise Decentralization

State Explosion

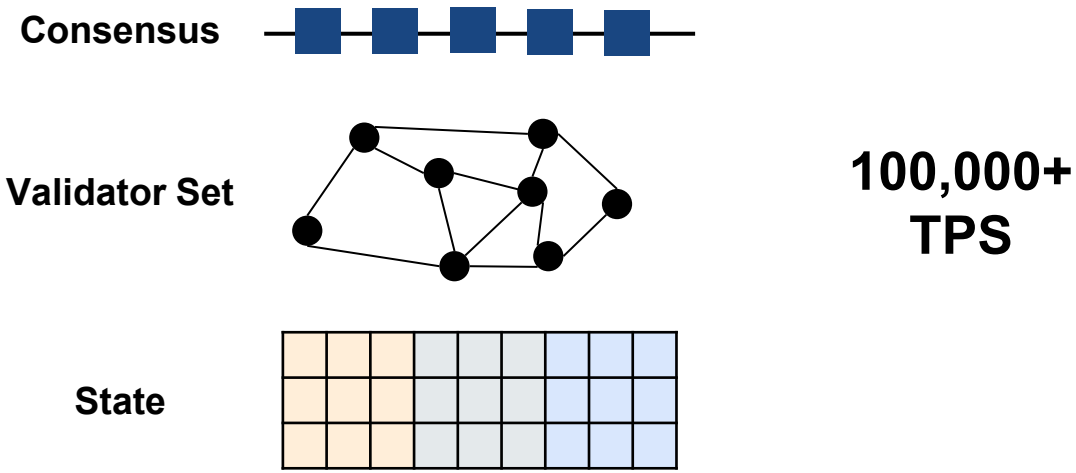
- e.g., Sui's Fullnode Storage Grows ~40GB per Day



Scalability Ceiling!

The Solution for Scalability Ceiling: The Sharding

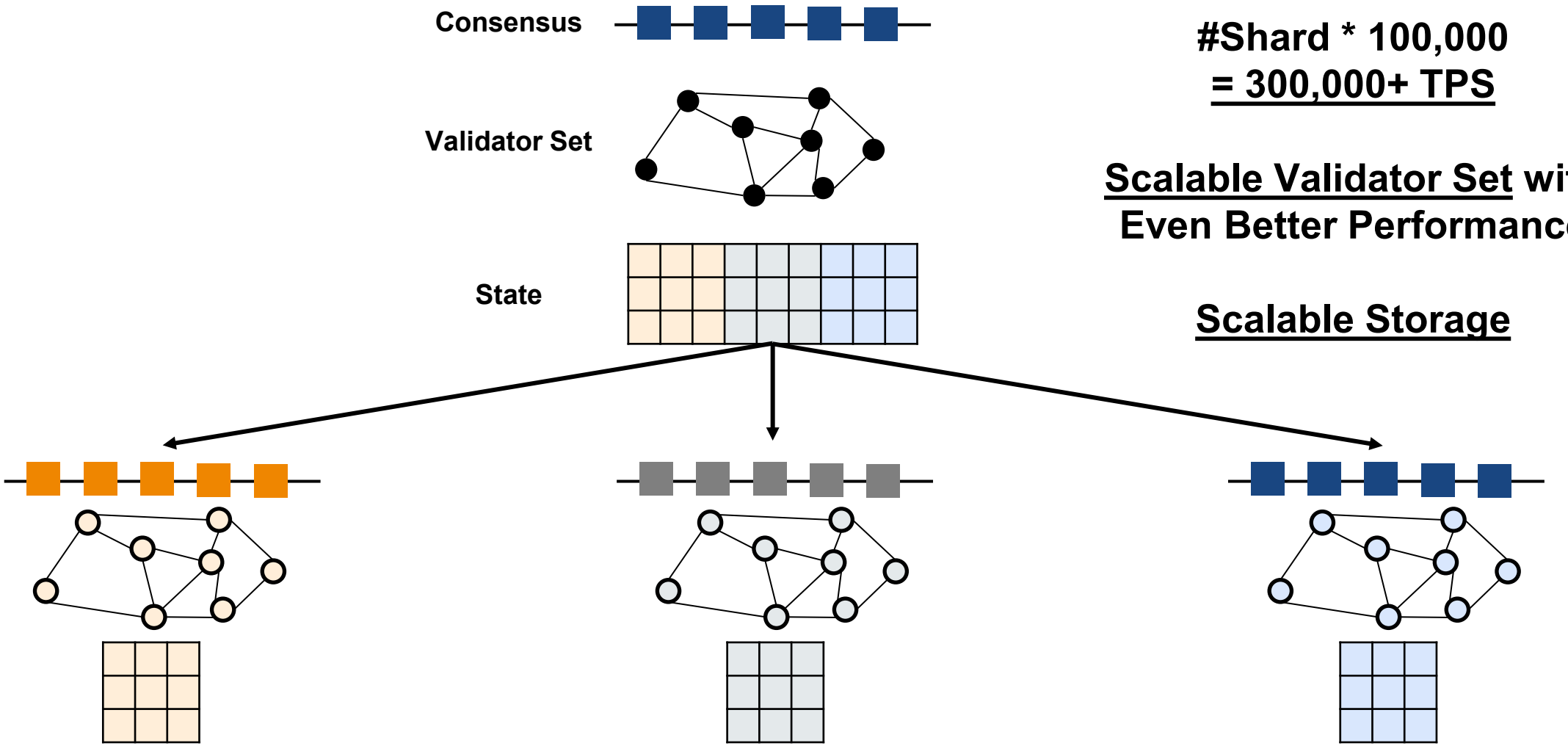
Sharding the High-Performance L1 Unlocks Scalability for Consensus, Validator Set, and Storage



100,000+ TPS

The Solution for Scalability Ceiling: The Sharding

Sharding the High-Performance L1 Unlocks Scalability for Consensus, Validator Set, and Storage



#Shard * 100,000
= 300,000+ TPS

Scalable Validator Set with Even Better Performance

Scalable Storage

Decipher

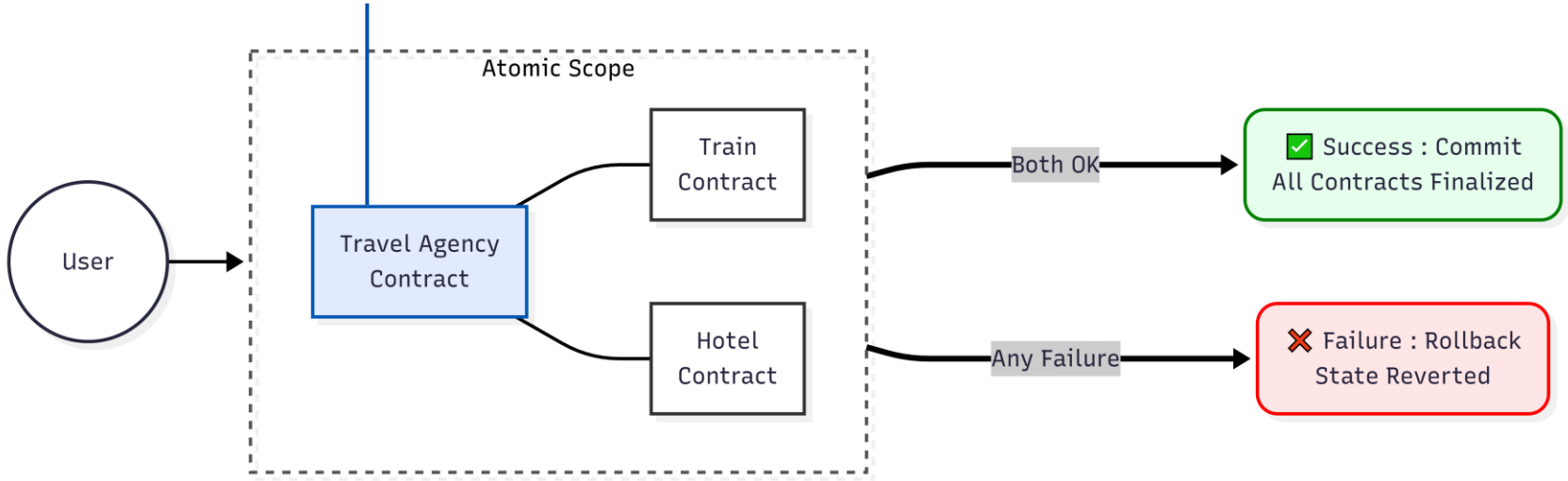
Sharding is No Silver Bullet

Resolving the Train-and-Hotel Problem is Pivotal for Sharding's Horizontal Scalability

Suppose a user wants to purchase a train ticket and reserve a hotel via a travel agency

We need to make sure that the operation is atomic - <u>either both reservations succeed or neither do</u>

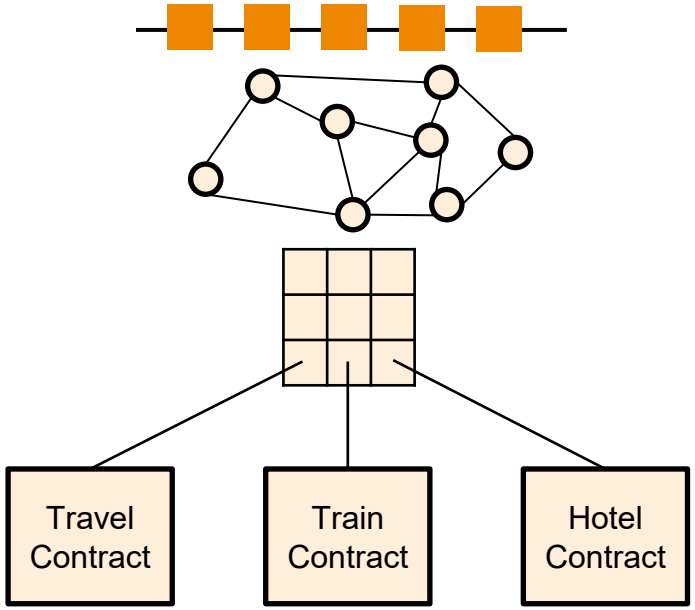
1. Check Train Seat Availability
2. Check Hotel Room Availability
3. Book Available Seat
4. Book Available Room



Decipher

Sharding is No Silver Bullet

If the Travel Agency, Train and Hotel Booking Contract Are on the Same Shard, This Is Easy



Single transaction that attempts to make both reservations through the travel agency contract is enough!

- Transaction calls the function of travel contract.
- Travel contract calls the train and hotel contract for availability checks and reservations.

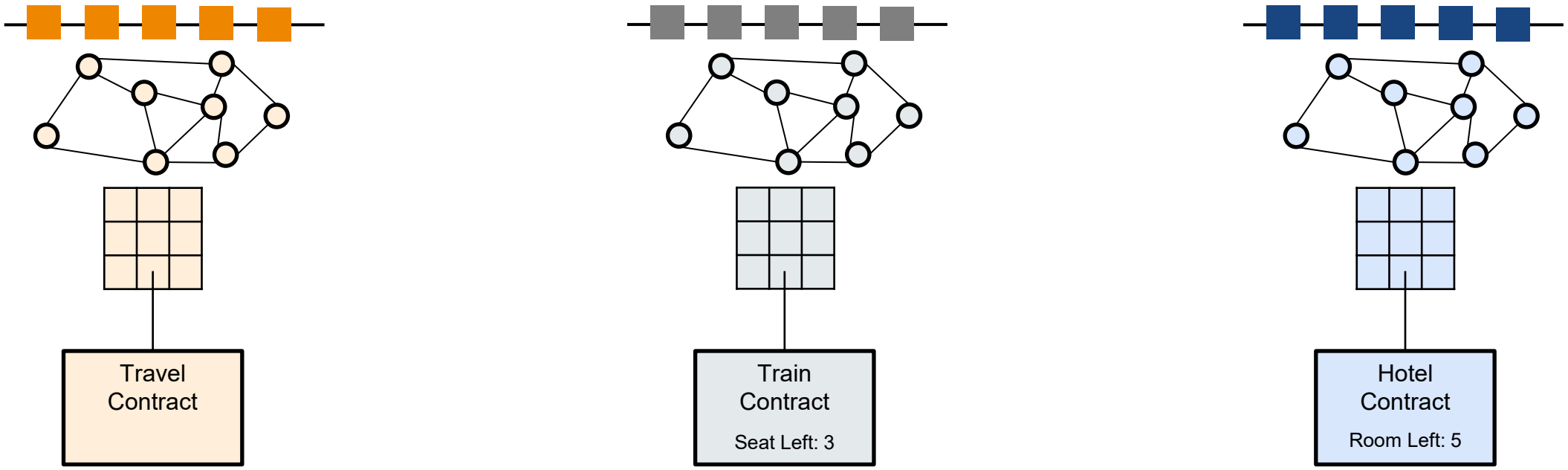
Transaction Latency: Single block time

4. The Atomicity Dilemma: The Train-And-Hotel Problem

Decipher

Sharding is No Silver Bullet

If The Contracts Are On Different Shards, This Is Where It Gets Complicated



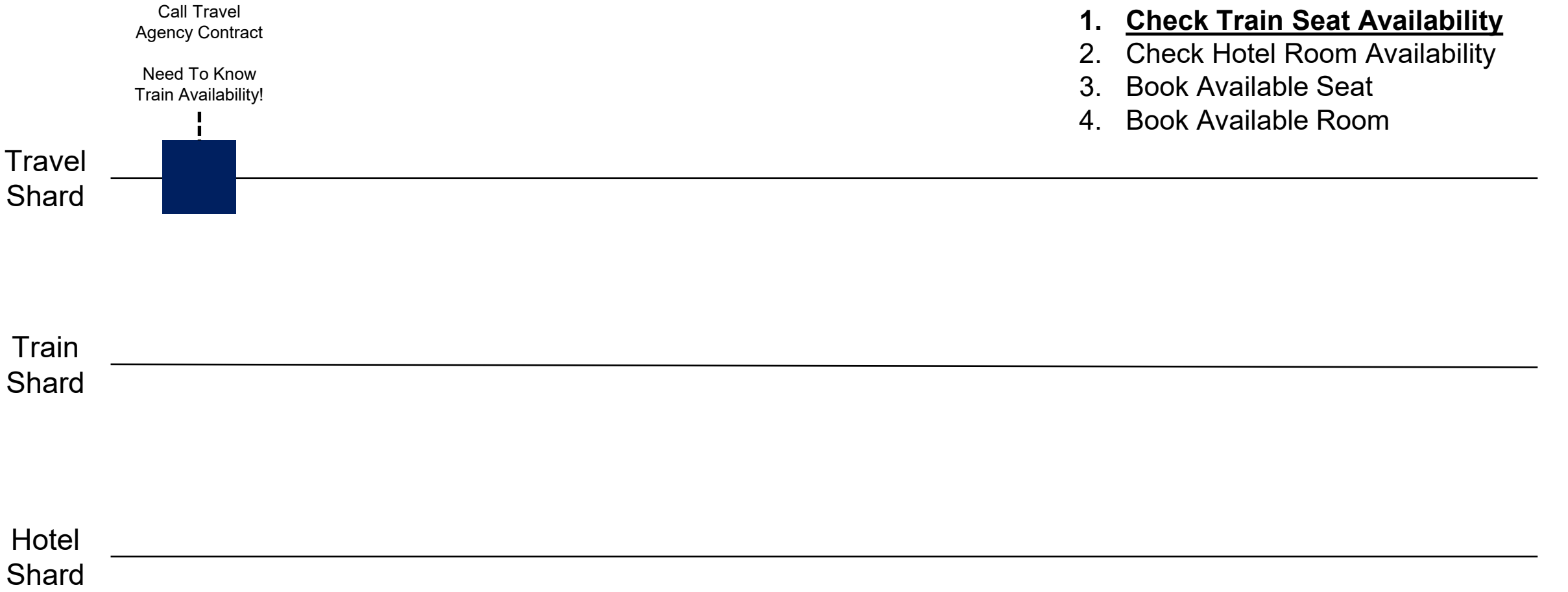
Ensuring atomicity (either both reservations succeed or neither do) of such transaction is inherently challenging due to the isolation of state spaces and the inability to synchronously share state across shards

Transaction Latency: Multiple block time by complex sharding protocol overhead

Decipher

How Current Sharding Blockchains Handles The Train-And-Hotel Problem

Execution Forwarding Protocol



Decipher

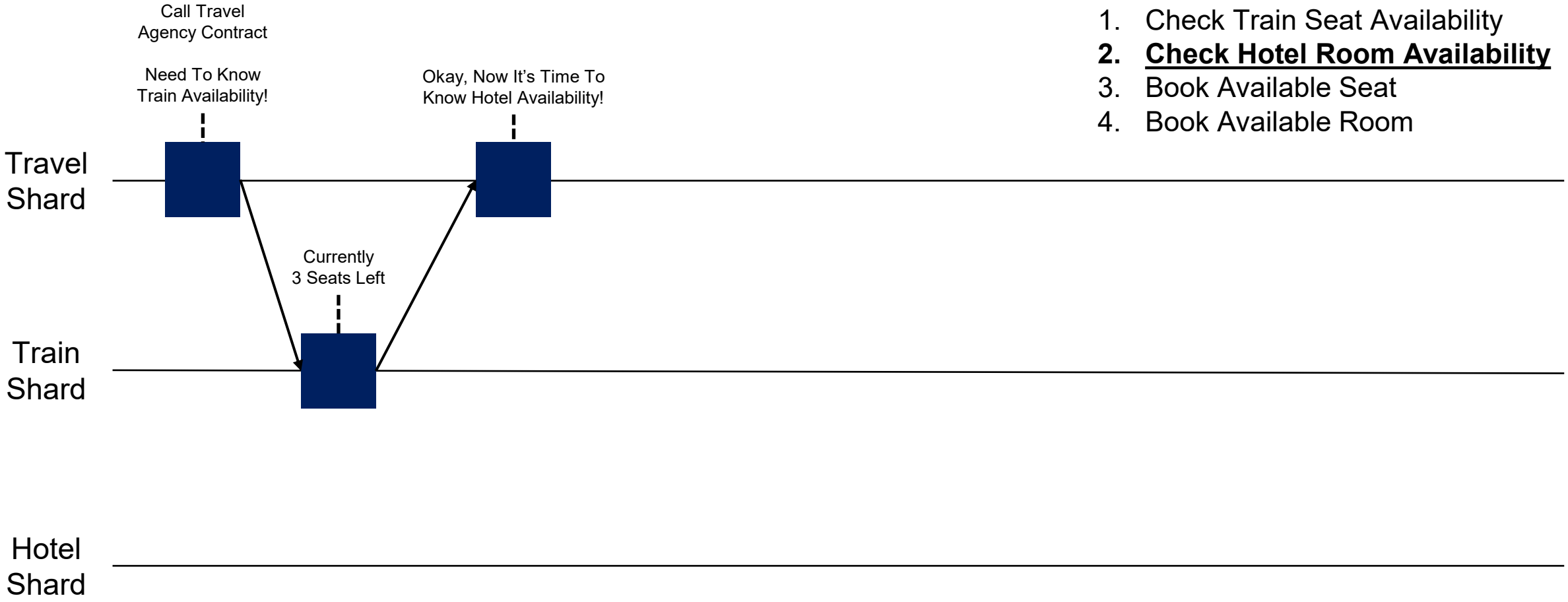
How Current Sharding Blockchains Handles The Train-And-Hotel Problem

Execution Forwarding Protocol

Decipher

How Current Sharding Blockchains Handles The Train-And-Hotel Problem

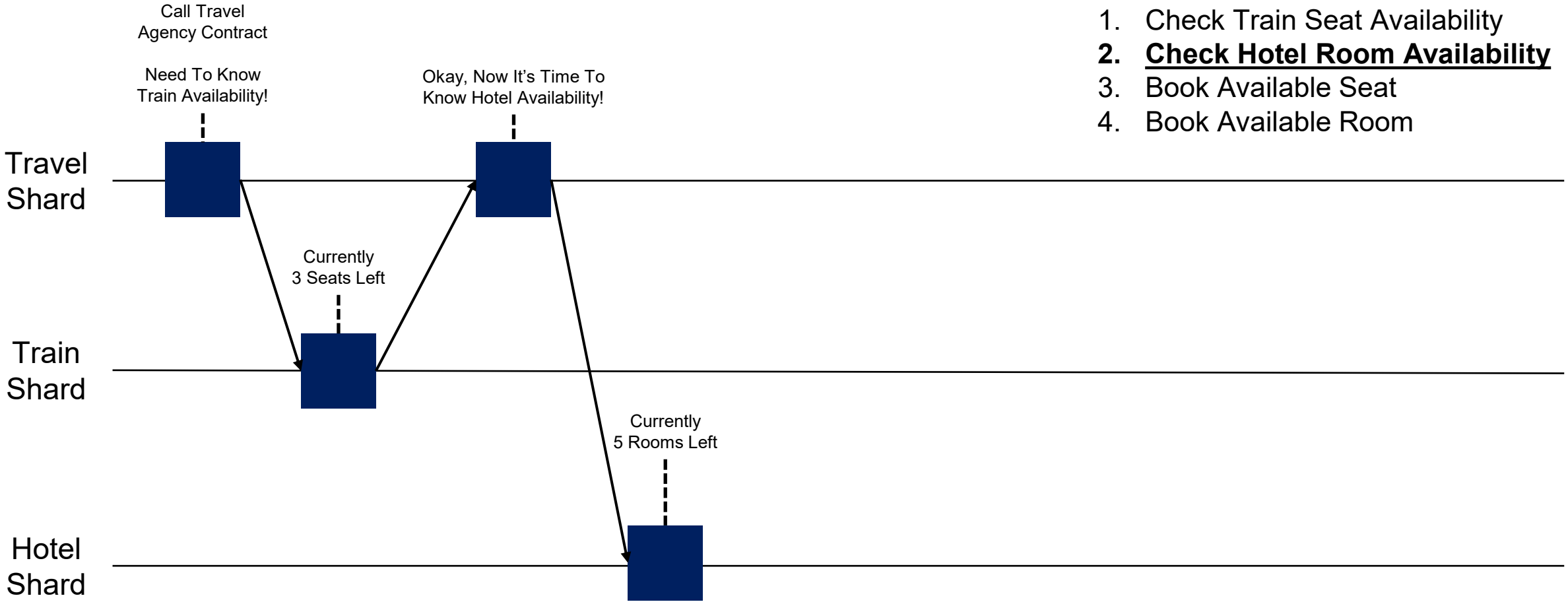
Execution Forwarding Protocol



Decipher

How Current Sharding Blockchains Handles The Train-And-Hotel Problem

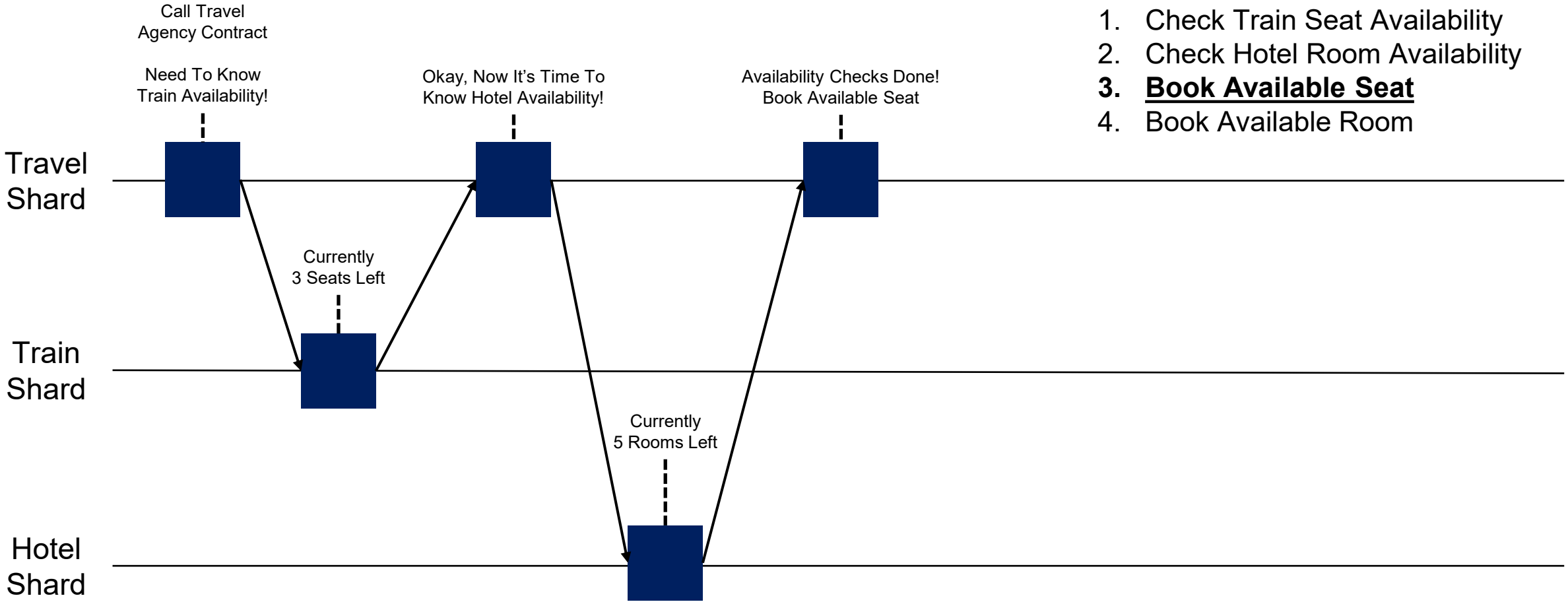
Execution Forwarding Protocol



Decipher

How Current Sharding Blockchains Handles The Train-And-Hotel Problem

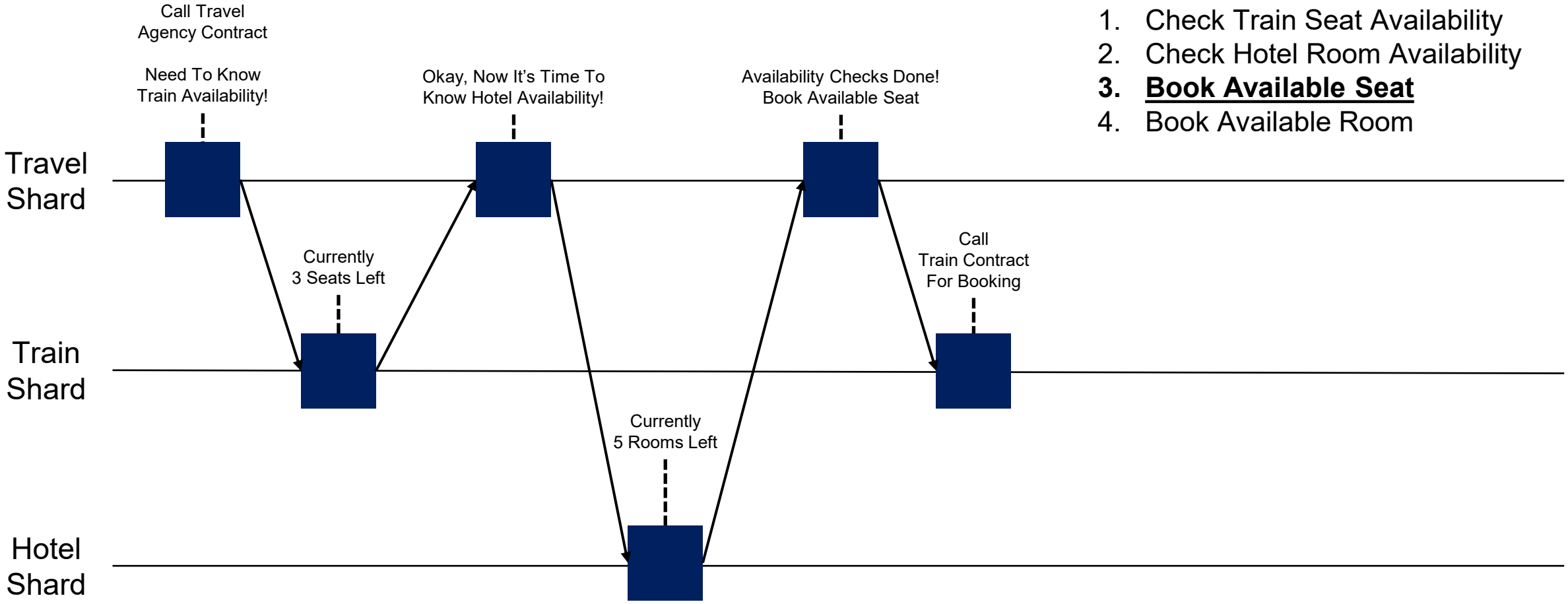
Execution Forwarding Protocol



Decipher

How Current Sharding Blockchains Handles The Train-And-Hotel Problem

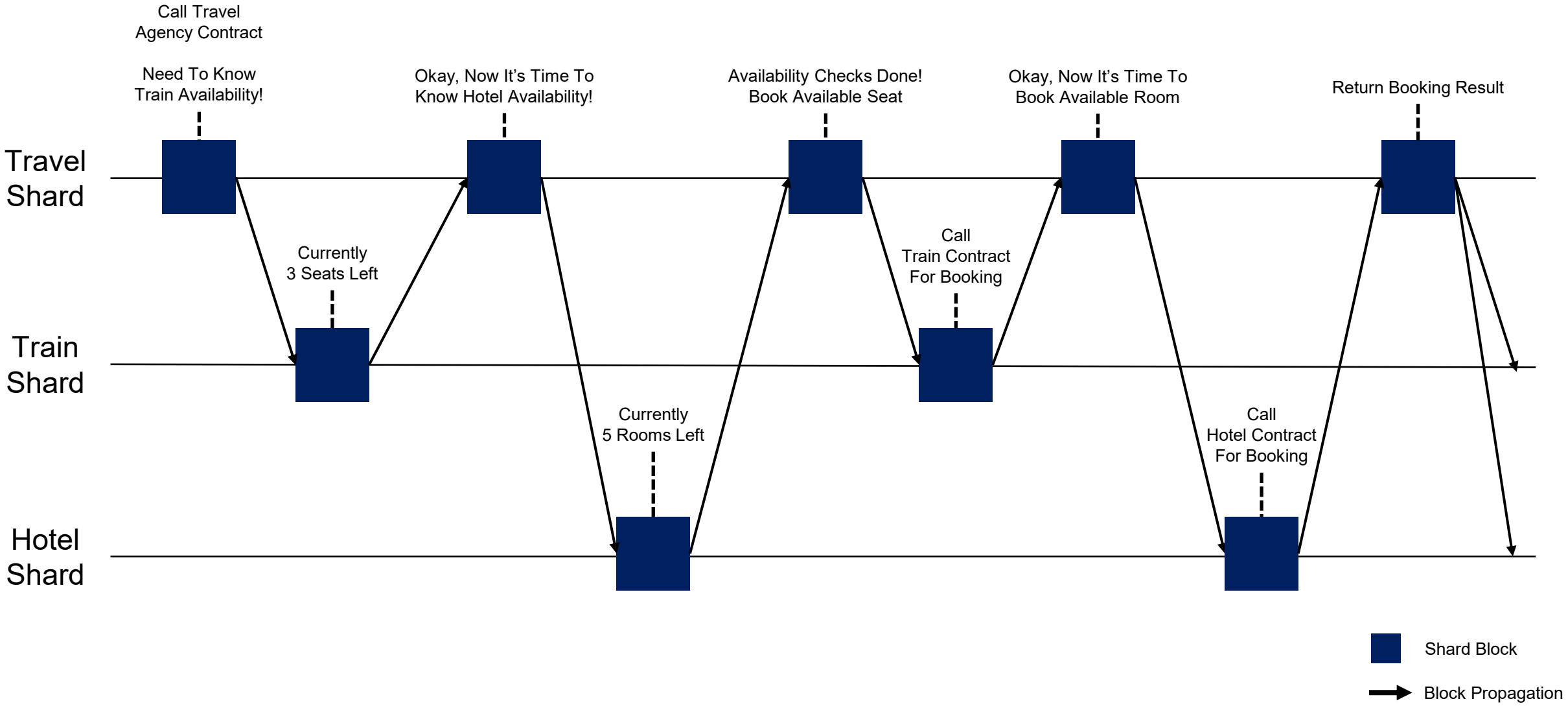
Execution Forwarding Protocol



Decipher

How Current Sharding Blockchains Handles The Train-And-Hotel Problem

Execution Forwarding Protocol, And So On

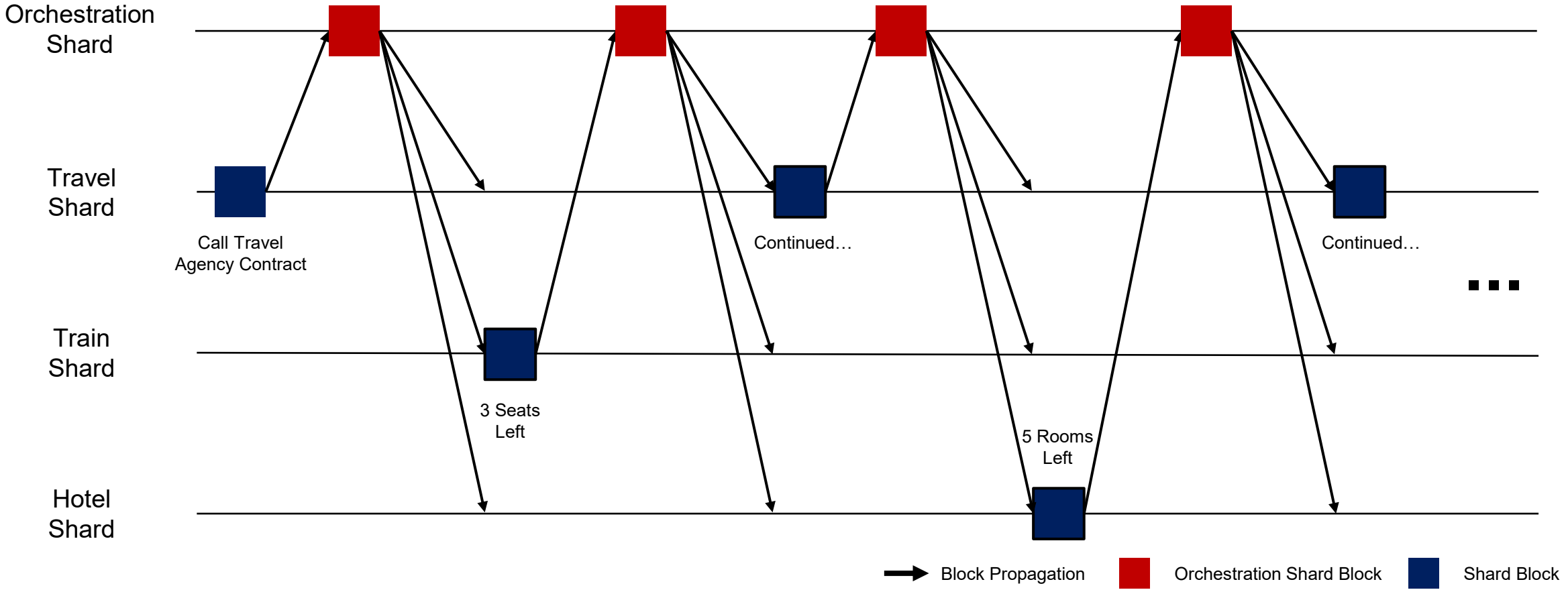


Decipher

How Current Sharding Blockchains Handles The Train-And-Hotel Problem

Actually, It Is Worse Than That

Additional consensus step for the Orchestration Shard Block is required at each stage to coordinate inter-shard communication and determine the ordering of cross-shard messages



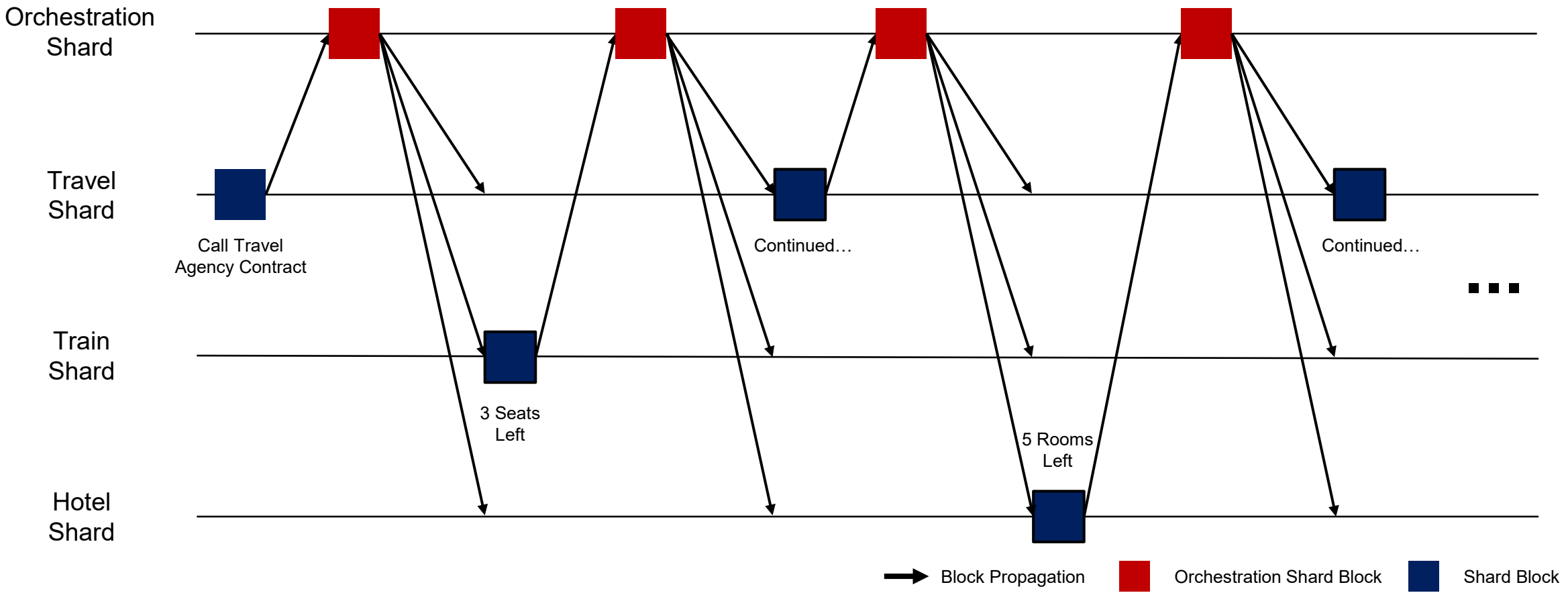
Decipher

How Current Sharding Blockchains Handles The Train-And-Hotel Problem

Actually, It Is Worse Than That

Transaction Latency: block time * 17

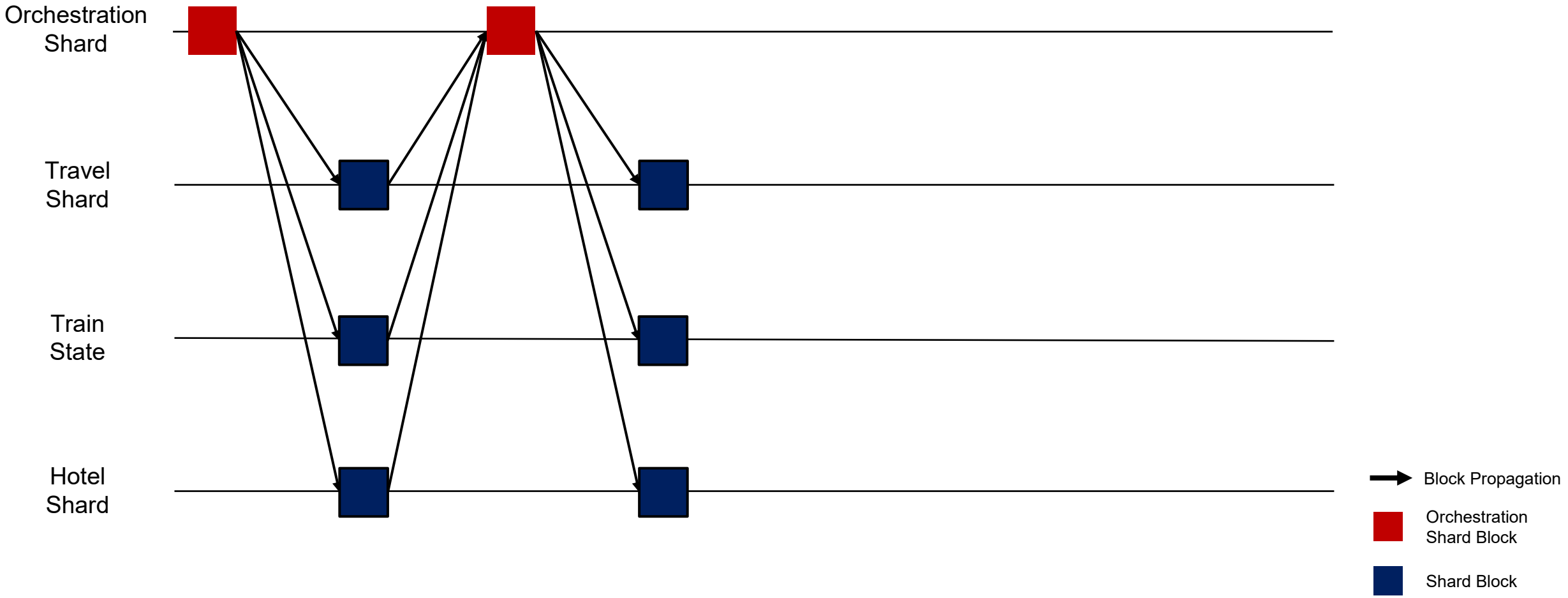
Longer Call Tree? Worse That This



Decipher

Matrix Guarantees Cross-Shard Transaction Finality With 4 Block Times

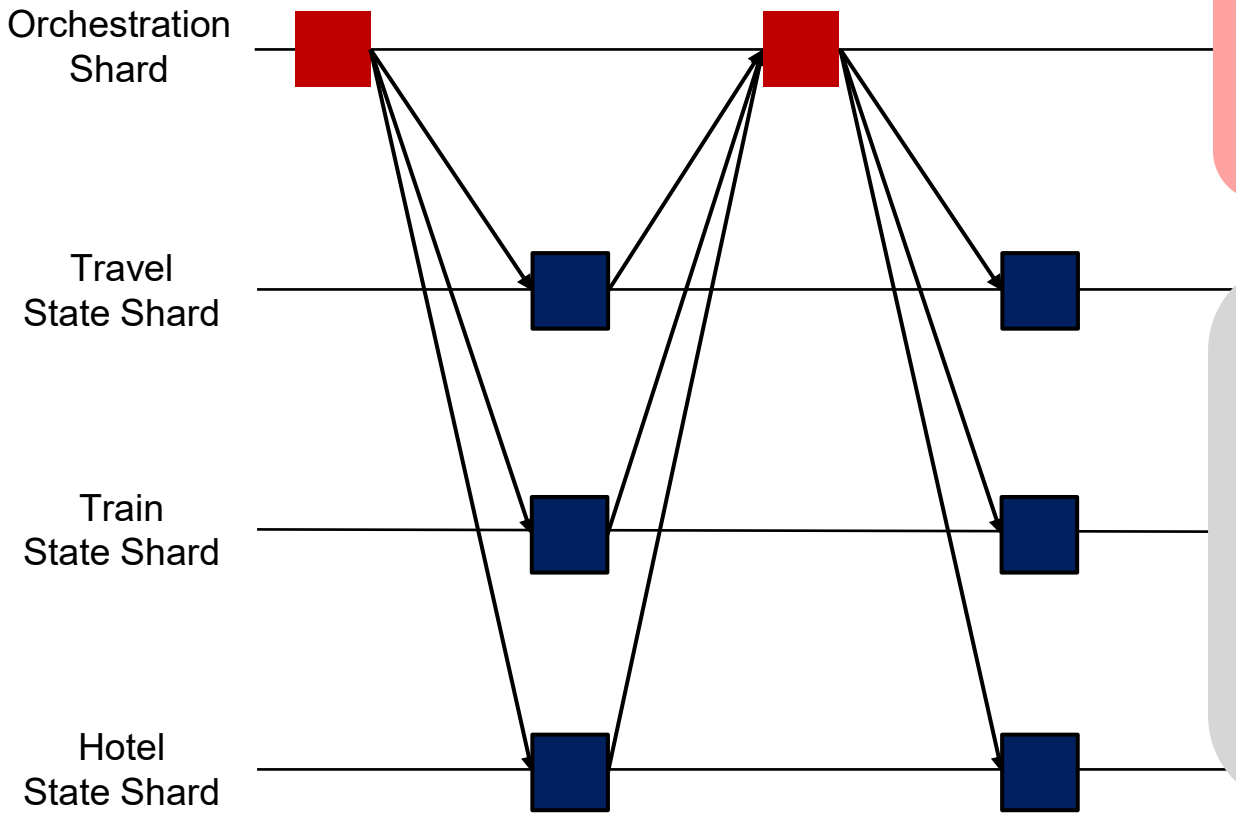
Long Call Tree? Still 4 Block Times



Decipher

Matrix Guarantees Cross-Shard Transaction Finality With 4 Block Times

The Architecture of Matrix



Orchestration Shard

- Orchestrate Cross-Shard Transactions Via Transaction Simulation
- No Explicit State, Only The State Root Of Each State Shards
- Maintain Contract Codes Of All State Shards

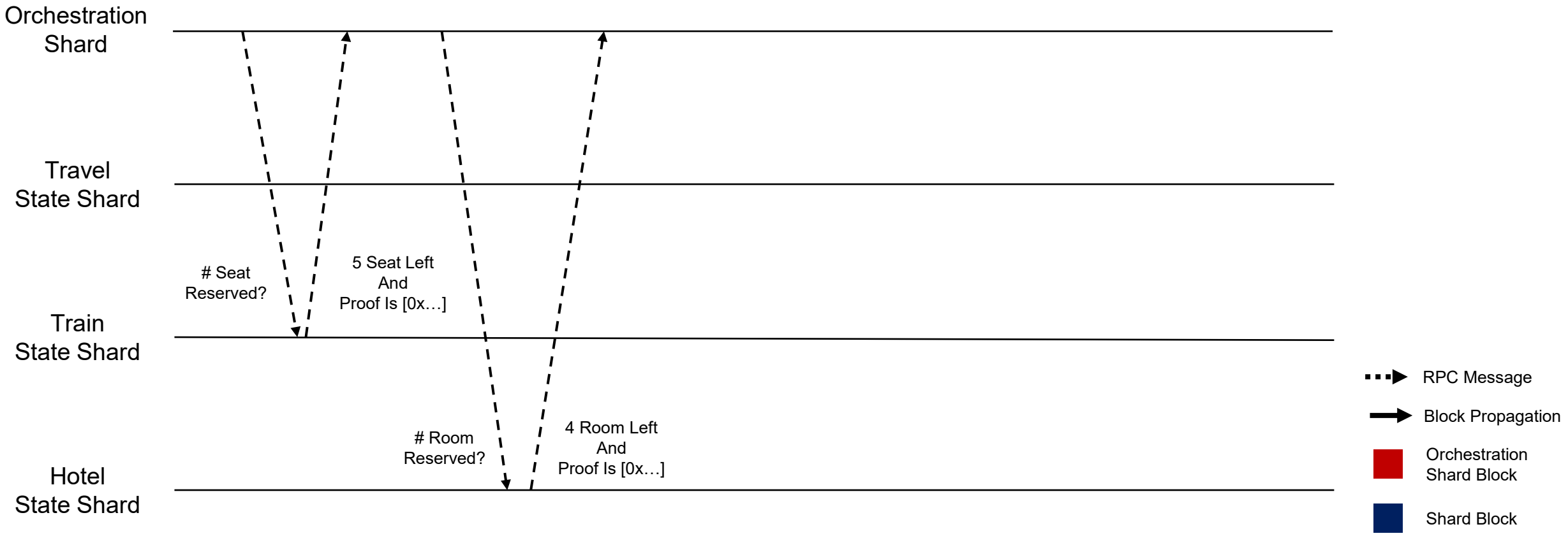
State Shard

- Maintain Explicit State
- Execute Local Transaction
- Apply Results Of Cross-Shard Transaction Orchestration Received From Orchestration Shard Block

Matrix Guarantees Cross-Shard Transaction Finality With 4 Block Times

The Protocol of Matrix

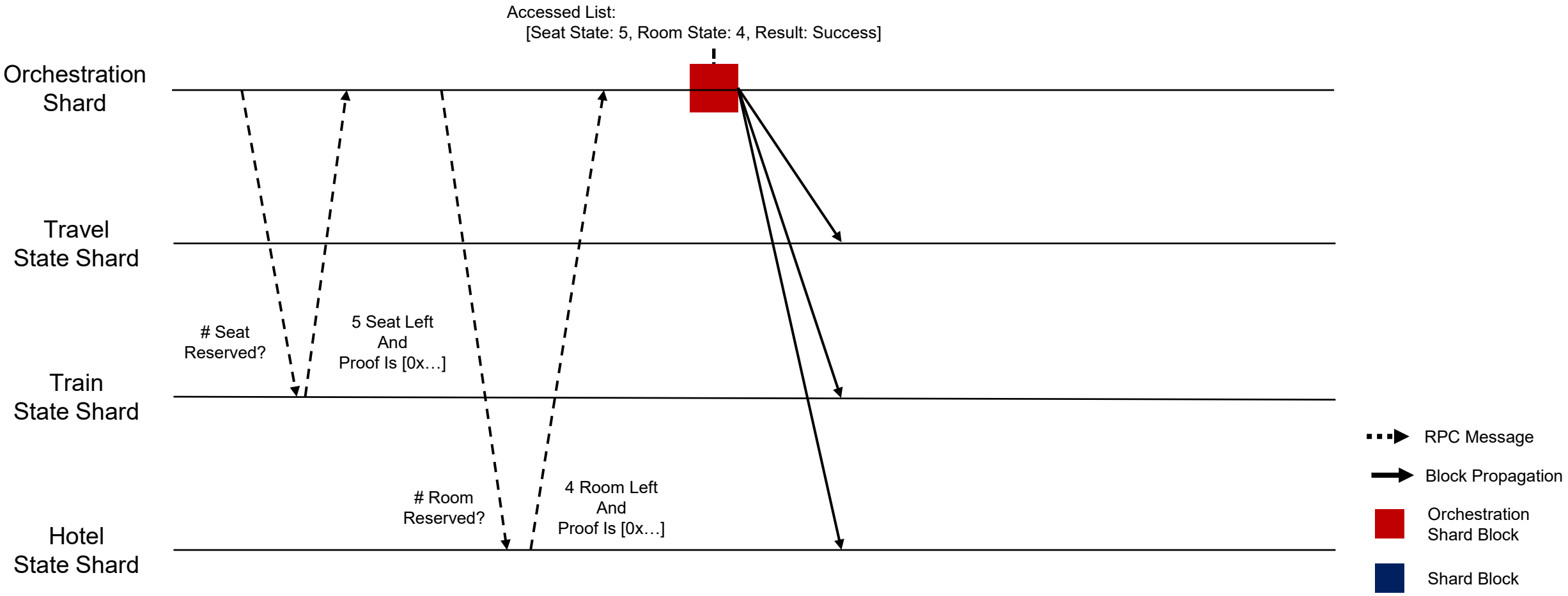
First, Cross-Shard Transaction Calling Travel Agency Contract Arrives At Orchestration Shard And Simulates The Execution By Requesting Missing States From State Shards With Proof!



Matrix Guarantees Cross-Shard Transaction Finality With 4 Block Times

The Protocol of Matrix

Then, Orchestration Shard Broadcasts Its Block With The Accessed State During The Simulation



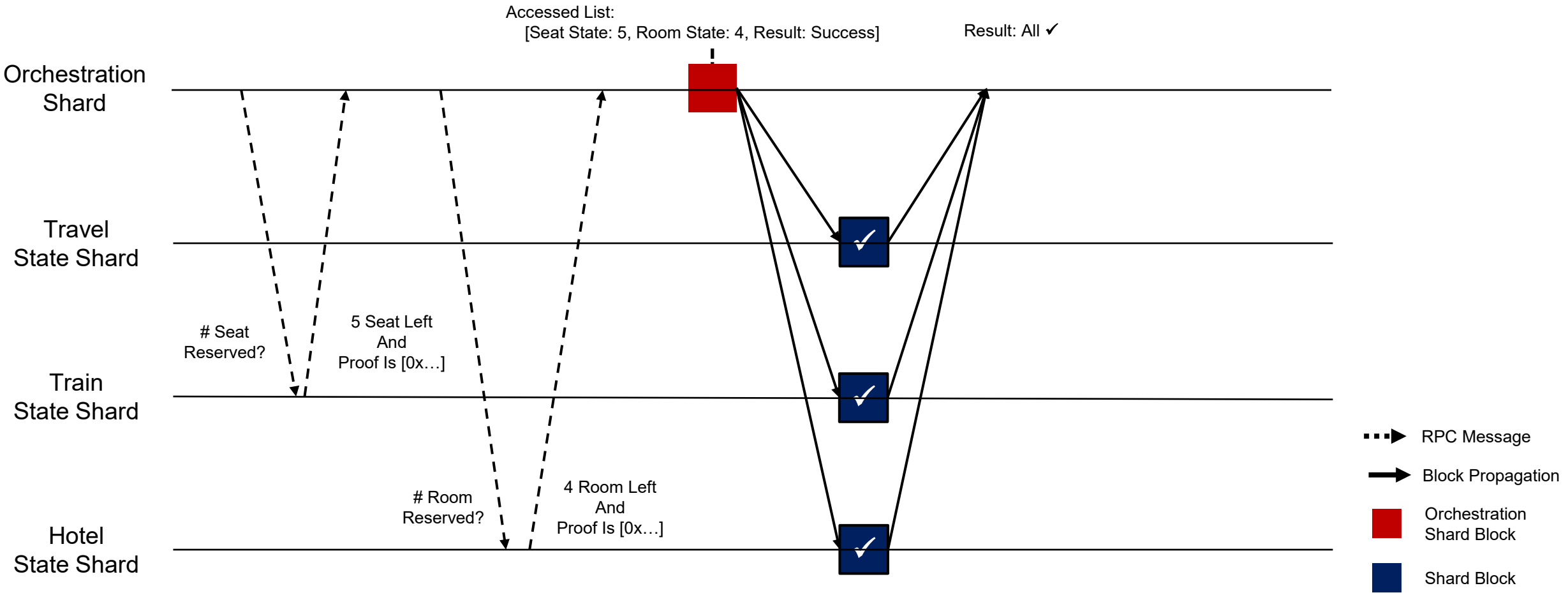
Decipher

Matrix Guarantees Cross-Shard Transaction Finality With 4 Block Times

The Protocol of Matrix

Next, State Shard That Received Orchestration Shard Block Checks The Accessed State During Simulation

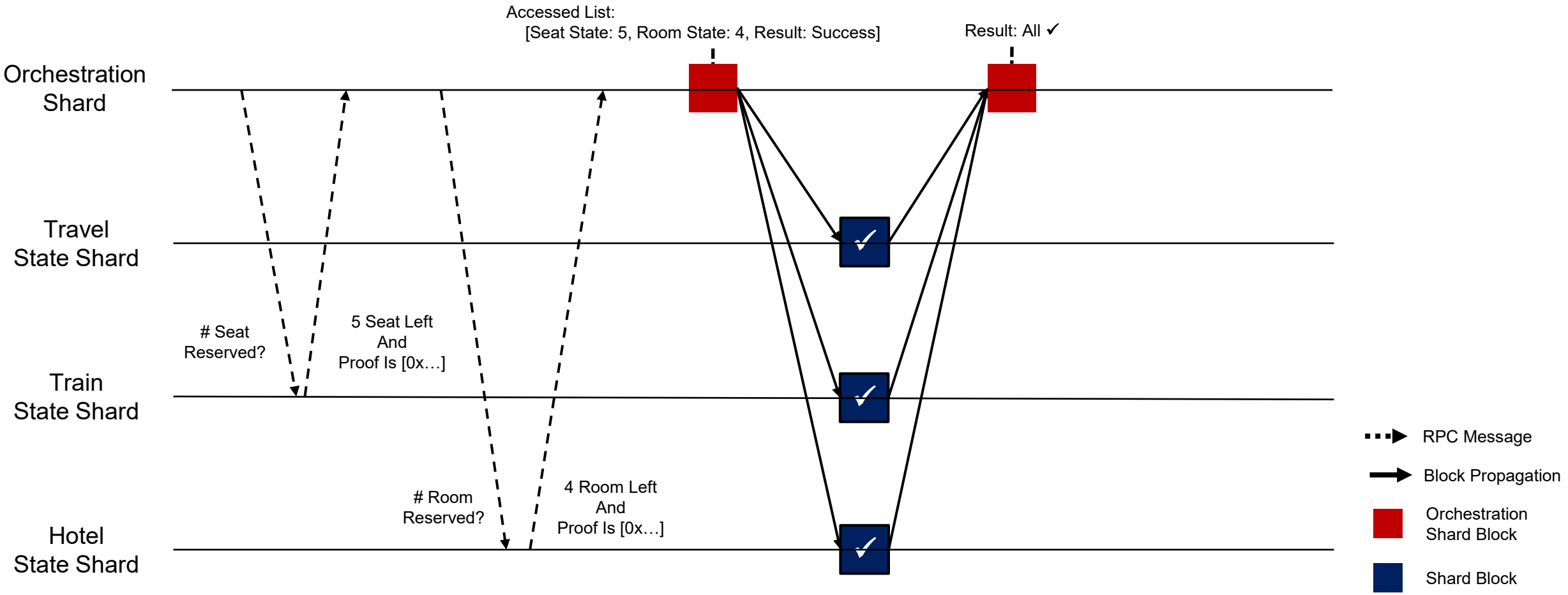
If It Is Not Locked, Lock The State And Vote To Commit In Its Block / If It Is Locked, Vote To Abort In Its Block



Matrix Guarantees Cross-Shard Transaction Finality With 4 Block Times

The Protocol of Matrix

Orchestration Shard Broadcasts Its Block With The Vote Of Each State Shard

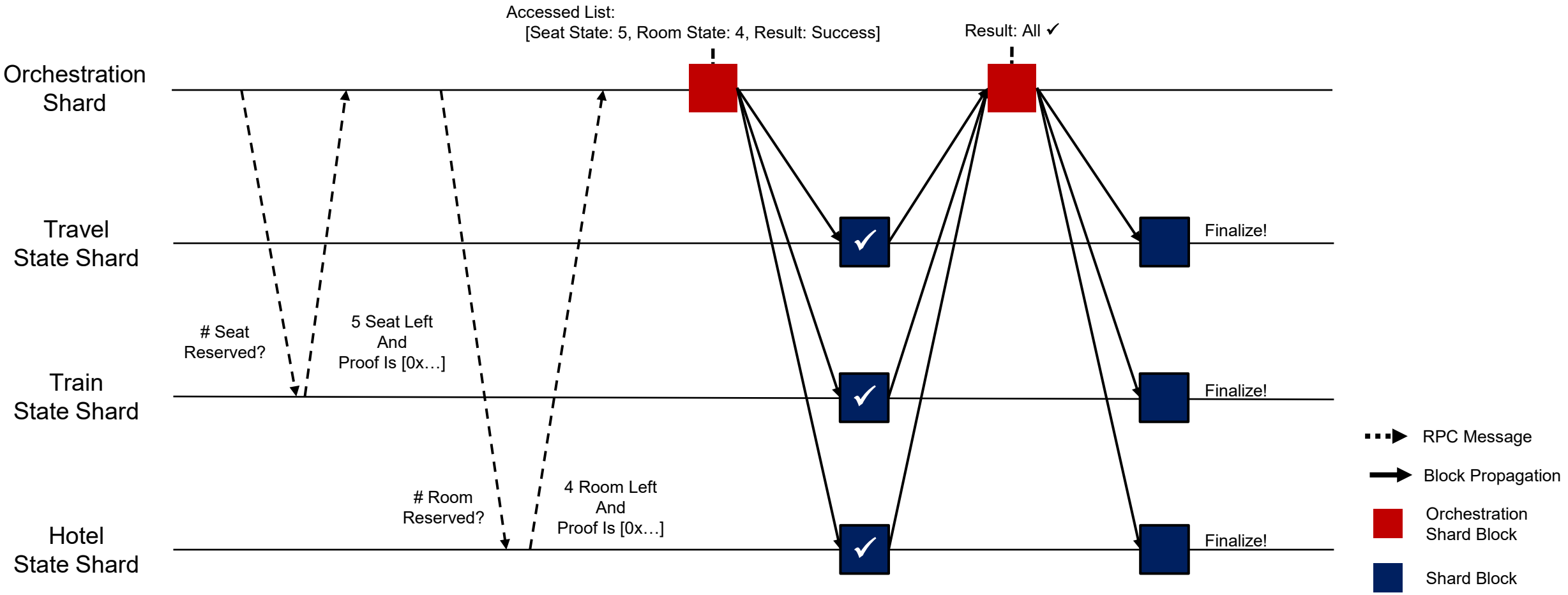


Decipher

Matrix Guarantees Cross-Shard Transaction Finality With 4 Block Times

The Protocol of Matrix

Upon Witnessing The Global Commit Votes Via Orchestration Shard, State Shard Applies Simulation Result And Releases Corresponding Locks To Finalize The Cross-Shard Transaction



Matrix Guarantees Cross-Shard Transaction Finality With 4 Block Times

The Evaluation of Matrix v.s. Baseline Cross-Shard Protocol

Github Repository: https://github.com/The-Sharding-Resurrection/test_v1

Matrix Guarantees Cross-Shard Transaction Finality With 4 Block Times

The Evaluation of Matrix v.s. Baseline Cross-Shard Protocol

Decipher

The Matrix: Sharding Resurrection

Nobody Is Really Talking About Sharding Anymore, But One Day…

For Teams Who Desire Unparallel Performance

For Teams Who Value The Decentralization The Most, But Do Not Want To Give Up The Performance

For Teams Who Belatedly Found Out That Their Node Storage Had Surpassed Petabyte

They Will Come Back To Sharding, With A Scalable Cross-Shard Protocol “Matrix”

Sharding Will Be Back